for short-lived functions, but the previous behavior is better suited to computationally intensive, longer-running functions. Set `JAVA_TOOL_OPTIONS` to `-XX:-TieredCompilation` to revert to the Java 11 behavior. For more information about the `JAVA_TOOL_OPTIONS` parameter, see the section called "Understanding the `JAVA_TOOL_OPTIONS` environment variable".

Kafka: Error handling and retry configuration issues

Issue: *Kafka event source mapping fails to configure retry settings or on-failure destinations*

Kafka retry configurations and on-failure destinations are only available for event source mappings with provisioned mode enabled. Ensure that you have configured `MinimumPollers` in your `ProvisionedPollerConfig` before attempting to set retry configurations.

Common configuration errors:

- **Infinite retries with bisect batch** – You cannot enable `BisectBatchOnFunctionError` when `MaximumRetryAttempts` is set to -1 (infinite). Set a finite retry limit or disable bisect batch.
- **Same topic recursion** – The Kafka on-failure destination topic cannot be the same as any of your source topics. Choose a different topic name for your dead letter topic.
- **Invalid Kafka destination format** – Use the `kafka://<topic-name>` format when specifying a Kafka topic as an on-failure destination.
- **kafka:WriteData permission issues** – Ensure your execution role has `kafka-cluster:WriteData` permissions for the destination topic. Topic doesn't exist timeout exceptions or write API throttling issues may require increasing the account limits.

Troubleshoot event source mapping issues in Lambda

Issues in Lambda that relate to an event source mapping can be more complex because they involve debugging across multiple services. Moreover, event source behavior can differ based on the exact event source used. This section lists common issues that involve event source mappings, and provides guidance on how to identify and troubleshoot them.

> **Note**
>
> This section uses an Amazon SQS event source for illustration, but the principles apply to other event source mappings that queue messages for Lambda functions.

Identifying and managing throttling

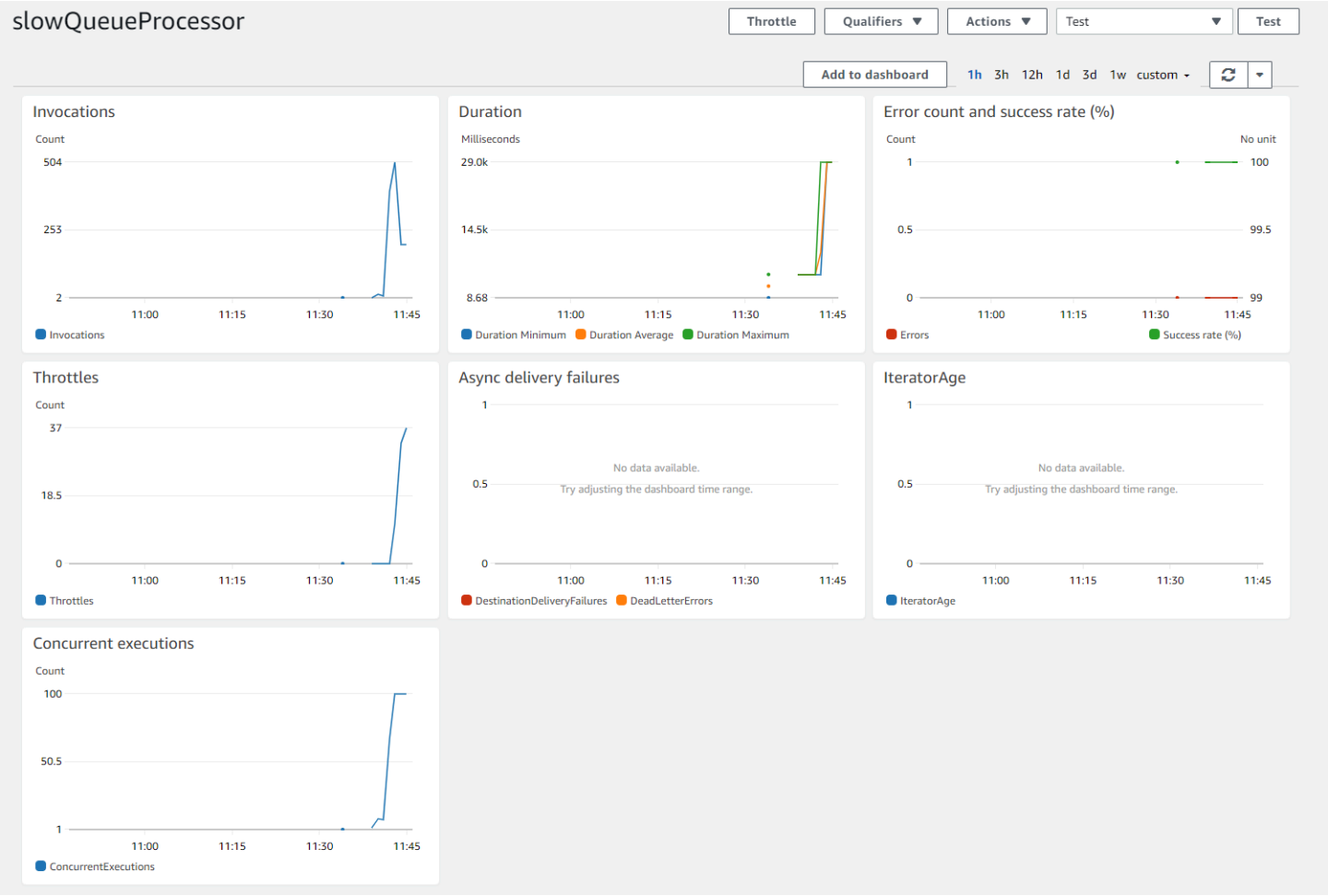
In Lambda, throttling occurs when you reach your function's or account's concurrency limit. Consider the following example, where there is a Lambda function that reads messages from an Amazon SQS queue. This Lambda function simulates 30 second invocations, and has a batch size of 1. This means that the function processes only 1 message every 30 seconds:

```
const doWork = (ms) => new Promise(resolve => setTimeout(resolve, ms))

exports.handler = async (event) => {
    await doWork(30000)

}
```

With such a long invocation time, messages begin arriving in the queue more rapidly than they are processed. If your account's unreserved concurrency is 100, Lambda scales up to 100 concurrent executions, and then throttling occurs. You can see this pattern in the CloudWatch metrics for the function:



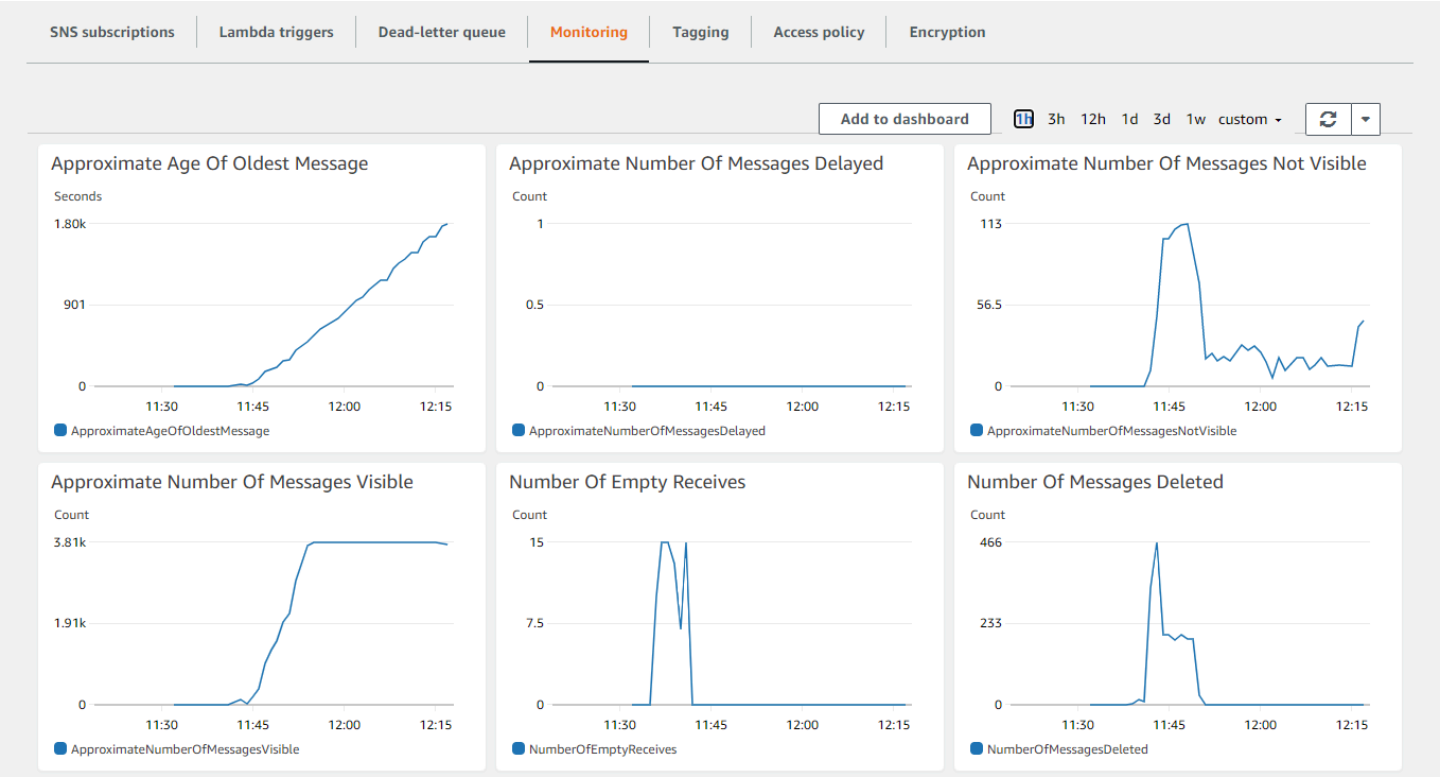
CloudWatch metrics for the function show no errors, but the **Concurrent executions** chart shows that the maximum concurrency of 100 is reached. As a result, the **Throttles** chart shows the throttling in place.

You can detect throttling with CloudWatch alarms, and setting an alarm anytime the throttling metric for a function is greater than 0. After you've identified the throttling issue, you have a few options for resolution:

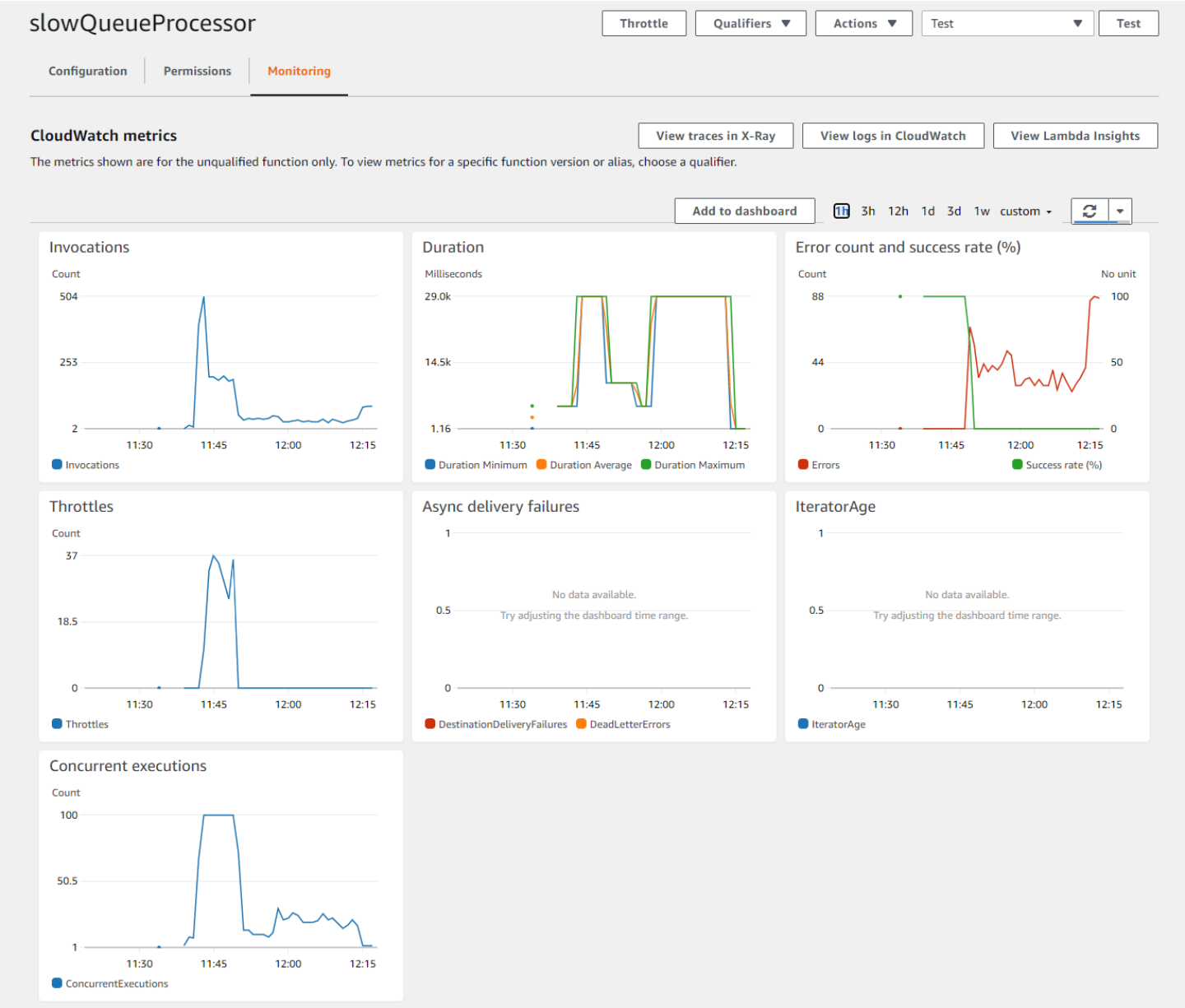
- Request a concurrency increase from AWS Support in this Region.
- Identify performance issues in the function to improve the speed of processing and therefore improve throughput.
- Increase the batch size of the function, so more messages are processed by each invocation.

Errors in the processing function

If the processing function throws errors, Lambda returns the messages to the SQS queue. Lambda prevents your function from scaling to prevent errors at scale. The following SQS metrics in CloudWatch indicate an issue with queue processing:

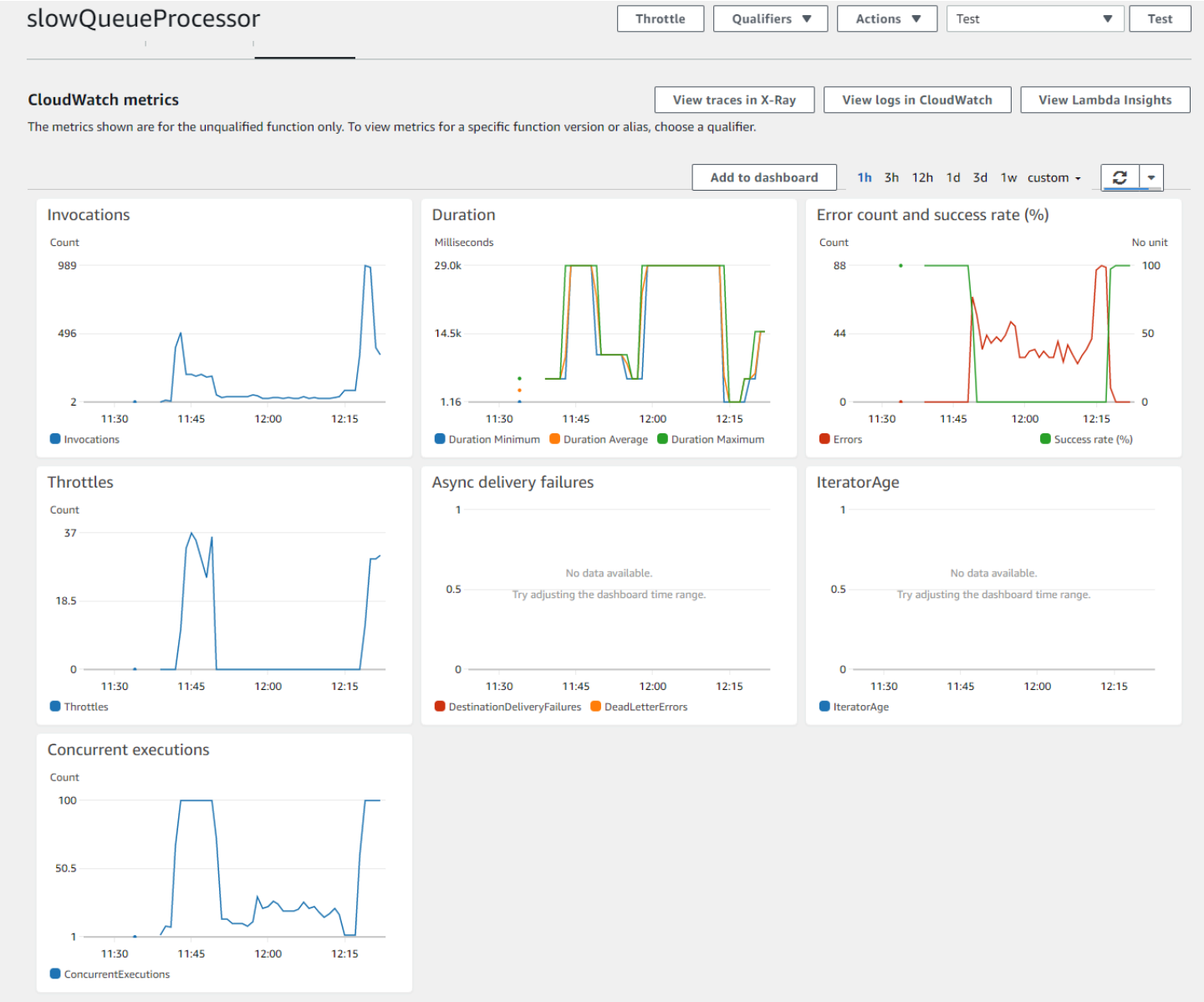


In particular, both the age of the oldest message and the number of messages visible are increasing, while no messages are deleted. The queue continues to grow but messages are not being processed. The CloudWatch metrics for the processing Lambda function also indicate that there is a problem:



The **Error count** metric is non-zero and growing, while **Concurrent executions** have reduced and throttling has stopped. This shows that Lambda has stopped scaling up your function due to errors. The CloudWatch logs for the function provide details of the type of error.

You can resolve this issue by identifying the function causing the error, then finding and resolving the error. After you fix the error and deploy the new function code, the CloudWatch metrics should show the processing recover:



Here, the **Error count** metric drops to zero and the **Success rate** metric returns to 100%. Lambda starts scaling up the function again, as shown in the **Concurrent executions** graph.

Identifying and handling backpressure

If an event producer consistently generates messages for an SQS queue faster than a Lambda function can process them, backpressure occurs. In this case, SQS monitoring should show the age of the oldest message growing linearly, along with the approximate number of messages visible. You can detect backpressure in queues using CloudWatch alarms.

The steps to resolve backpressure depend on your workload. If the primary goal is to increase processing capability and throughput by the Lambda function, you have a few options: